Practical 1 — Build & populate the shopdb database

1. Create a database named shopdb.

```
CREATE DATABASE shopdb;
USE shopdb;
```

2. Create the five tables from the schema specification, choosing appropriate data types and applying every listed constraint (including the composite primary key on order_items).

```
CREATE TABLE customers (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  city VARCHAR(50),
  signup_date DATE NOT NULL DEFAULT (CURRENT_DATE)
);
CREATE TABLE categories (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE products (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  category_id INT,
  price DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL DEFAULT 0,

  CONSTRAINT fk_product_category
    FOREIGN KEY (category_id)
    REFERENCES categories(category_id),

  CONSTRAINT chk_product_price
    CHECK (price > 0),

  CONSTRAINT chk_product_stock
    CHECK (stock >= 0)
);
```

```
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  order_date DATE NOT NULL,
  status VARCHAR(20) DEFAULT 'placed',

  CONSTRAINT fk_order_customer
    FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id),

  CONSTRAINT chk_order_status
    CHECK (status IN ('placed', 'shipped', 'delivered', 'cancelled'))
);
CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  quantity INT,
  unit_price DECIMAL(10,2) NOT NULL,

  PRIMARY KEY (order_id, product_id),

  CONSTRAINT fk_item_order
    FOREIGN KEY (order_id)
    REFERENCES orders(order_id),

  CONSTRAINT fk_item_product
    FOREIGN KEY (product_id)
    REFERENCES products(product_id),

  CONSTRAINT chk_item_quantity
    CHECK (quantity > 0)
);

# 3. Insert all Appendix A sample data, in an order that respects the foreign keys.

INSERT INTO categories (category_name)
VALUES
('Electronics'),
('Books'),
('Clothing');
```

```
INSERT INTO customers
(name, email, city, signup_date)
VALUES
('Asha Rao', 'asha@x.com', 'Pune', '2024-01-05'),
('Ravi Nair', 'ravi@x.com', 'Mumbai', '2024-02-10'),
('Sara Ali', 'sara@x.com', 'Pune', '2024-03-01'),
('Om Verma', 'om@x.com', 'Delhi', '2024-03-15'),
('Neha Shah', 'neha@x.com', 'Mumbai', '2024-04-02');

INSERT INTO products
(product_id, product_name, category_id, price, stock)
VALUES
(1, 'Laptop', 1, 60000, 10),
(2, 'Headphones', 1, 2000, 50),
(3, 'SQL Book', 2, 800, 100),
(4, 'Novel', 2, 400, 60),
(5, 'T-Shirt', 3, 700, 80),
(6, 'Jacket', 3, 3000, 20);

INSERT INTO orders
(order_id, customer_id, order_date, status)
VALUES
(1, 1, '2024-05-01', 'delivered'),
(2, 2, '2024-05-03', 'shipped'),
(3, 1, '2024-06-10', 'placed'),
(4, 3, '2024-06-12', 'delivered'),
(5, 4, '2024-07-01', 'cancelled');

INSERT INTO order_items
(order_id, product_id, quantity, unit_price)
VALUES
(1, 1, 1, 60000),
(1, 2, 2, 2000),
(2, 3, 3, 800),
(2, 5, 2, 700),
(3, 2, 1, 2000),
(4, 1, 1, 60000),
(4, 6, 1, 3000),
(5, 4, 2, 400);

SELECT * FROM customers;
```

```
SELECT * FROM categories;
SELECT * FROM products;
SELECT * FROM orders;
SELECT * FROM order_items;

SELECT
  o.order_id,
  c.name AS customer,
  o.order_date,
  o.status,
  p.product_name,
  oi.quantity,
  oi.unit_price
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id
ORDER BY o.order_id;

# 4. Write and run three verification statements:

# (a) list every product with its category name and price;
SELECT
  p.product_name,
  c.category_name,
  p.price
FROM products p
JOIN categories c
ON p.category_id=c.category_id;
```

Output:

	product_name	category_name	price
▶	SQL Book	Books	800.00
	Novel	Books	400.00
	T-Shirt	Clothing	700.00
	Jacket	Clothing	3000.00
	Laptop	Electronics	60000.00
	Headphones	Electronics	2000.00

(b) reduce the stock of 'Laptop' by 1;
```
SET SQL_SAFE_UPDATES=0;

UPDATE products
SET stock = stock -1
WHERE product_name='Laptop';

SELECT product_name, stock
FROM products
WHERE product_name = 'Laptop';
```

Output:

	product_name	stock
▶	Laptop	9

(c) delete the cancelled order together with its order_items.
```
DELETE FROM order_items
WHERE order_id IN (
  SELECT order_id
  FROM orders
  WHERE status = 'cancelled'
);

DELETE FROM orders
WHERE status = 'cancelled';

# verifying
SELECT * FROM orders WHERE status = 'cancelled';
SELECT * FROM order_items WHERE order_id = 5;
```

Output:

	order_id	product_id	quantity	unit_price
*	NULL	NULL	NULL	NULL